SLIIT
Discover Your Future

IT1170 – Data Structures and Algorithms

Lecture 03

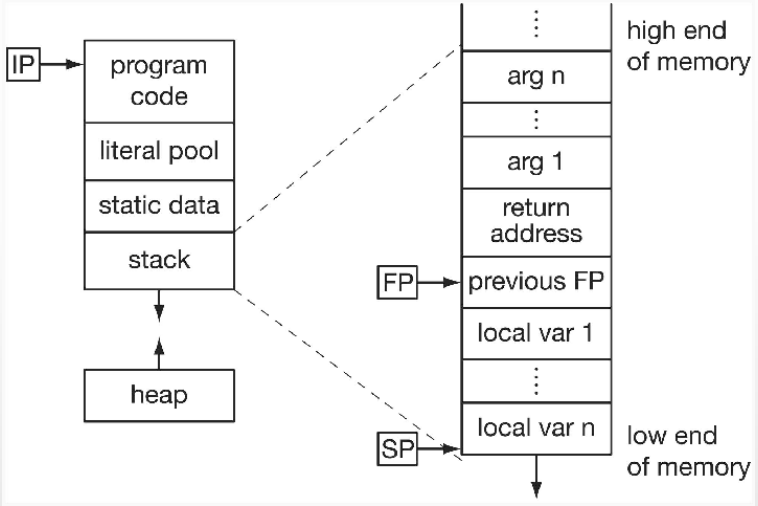
Introduction to Stack

Stack

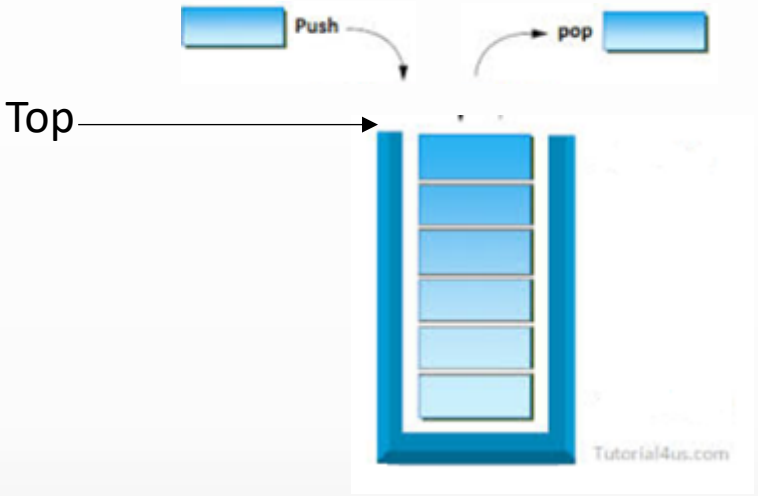
- Allows access to only one data item; the last item inserted
- If you remove this item, then you can access the next-to-last item inserted

Application of Stacks

- String Reverse
- Page visited history in Web browser.
- Undo sequence of text editor.
- Recursive function calling.
- Auxiliary data structure for Algorithms.
- Stack in memory for a process



Stack

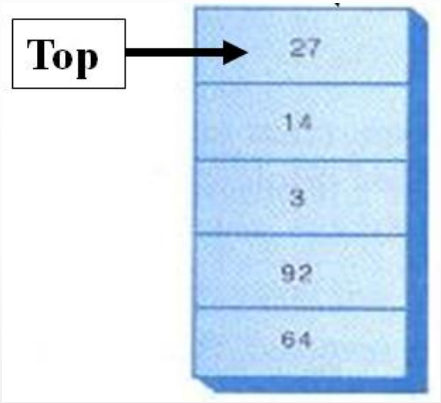


- In a stack all insertions and deletions are made at one end (Top). Insertions and deletions are restricted from the Middle and at the End of a Stack

- Adding an item is called Push
- Removing an item is called Pop

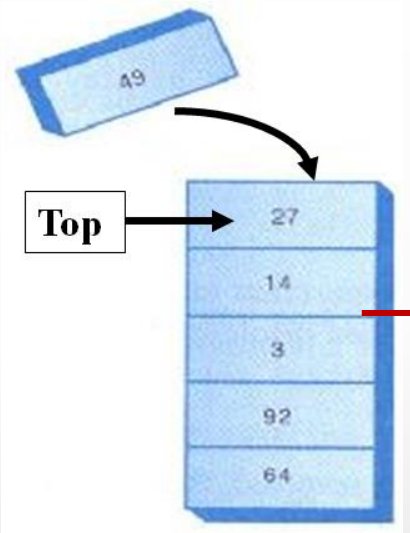
- Elements are removed from a Stack in the reverse order of that in which the elements were inserted into the Stack
- The elements are inserted and removed according to the Last-In-First-Out (LIFO) principle.

Stack - Push

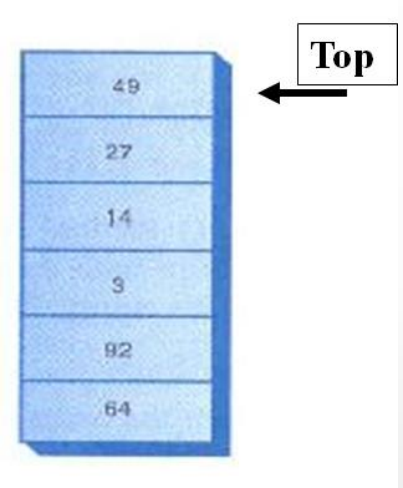


Before inserting

Push item 49

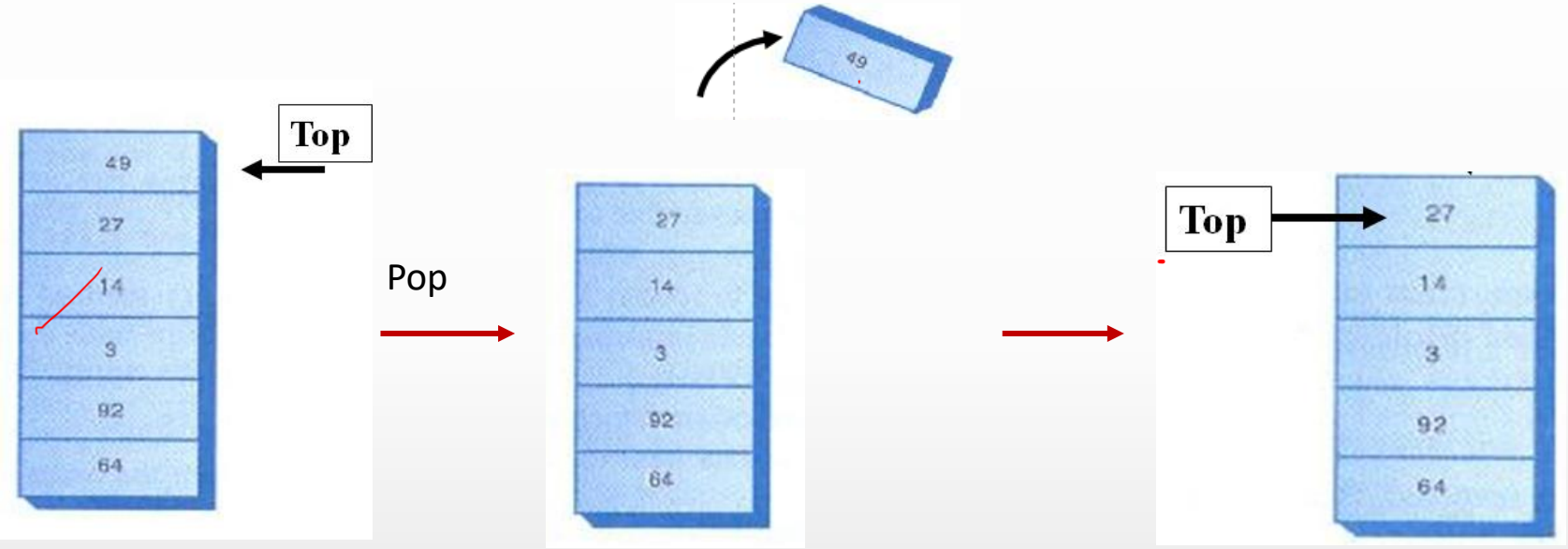


Item 49 is inserted on top



After inserting

Stack - Pop

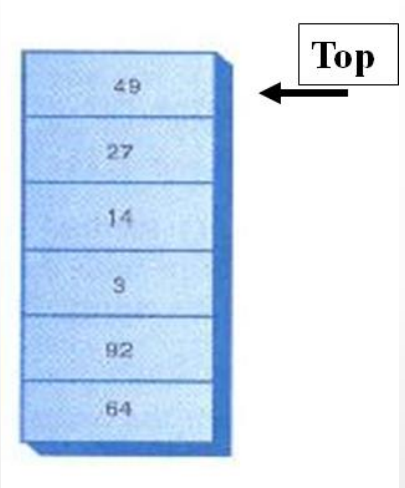


Before removing

Top item is removed

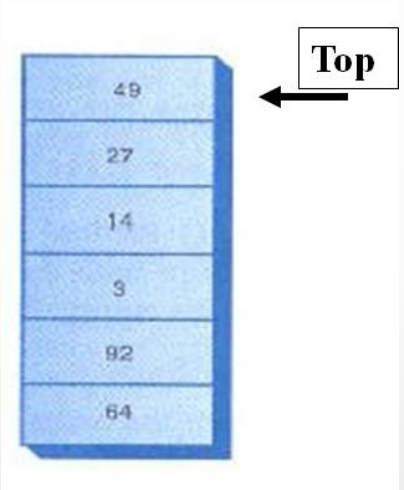
After removing

Stack - Peek



49

Peek

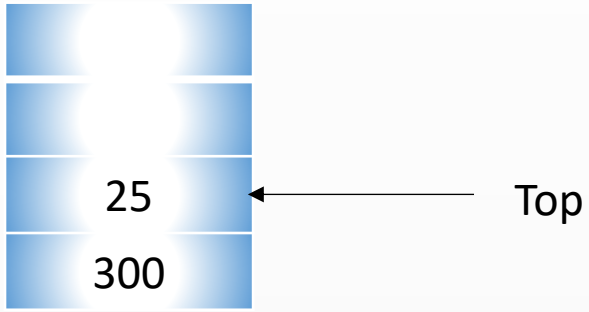


Stack remains the same

Peek is used to read the value from the top of the stack without removing it. You can peek only the Top item, all the other items are invisible to the stack user.

Question

Draw the stack frame after performing the below operations to the stack given below.



i) Push item 50
ii) Push item 500
iii) Peek
iv) Push item 100
v) Pop
vi) Pop
vii) Pop
viii) Pop

Uses of Stack

- The stack operations are built into the microprocessor.
- When a method is called, its return address and arguments are pushed onto a stack, and when it returns they're popped off.

Stack - Implementation

Stack implementation using an **array**

- Constructor creates a new stack of a size specified in its argument.

- Variable *top*, which stores the index of the item on the top of the stack.

```
class StackX {

    private int maxSize;    // size of stack array
    private double[] stackArray;
    private int top;        //top of the stack

    publc StackX(int s) {   // constructor

            maxSize = s;        // set array size
            stackArray = new double[maxSize];
            top = -1;           // no items
    }

    ......................
    ......................
}
```

Implementation - push

```
class StackX{

    private int maxSize;    // size of stack array
    private double[] stackArray;
    private int top;        //top of the stack

    publc StackX(int s) { // constructor

        maxSize = s;      // set array size
        stackArray = new double[maxSize];
        top = -1;         // no items
    }
    public void push(double j) {

        // increment top
        // insert item
    }
```

Implementation - push

```
class StackX {
        private int maxSize;    // size of stack array
        private double[] stackArray;
        private int top;          //top of the stack

        publc StackX(int s) {// constructor

                maxSize = s;       // set array size
                stackArray = new double[maxSize];
                top = -1;           // no items
        }
      public void push(double j) {

                // increment top.  insert item
                stackArray[++top] = j;
      }
```

What if the stack is full ?

Implementation - push

```
class StackX
{
    private int maxSize;    // size of stack array
    private double[] stackArray;
    private int top;        //top of the stack

    public StackX(int s) {// constructor

        maxSize = s;      // set array size
        stackArray = new double[maxSize];
        top = -1;         // no items
    }
    public void push(double j) {

        // check whether stack is full
        if (top == maxSize – 1)
            System.out.println("Stack is full");
        else
            stackArray[++top] = j;
    }
```

Implementation – pop/peek

```
class StackX
{
    private int maxSize;    // size of stack array
    private double[] stackArray;
    private int top;        //top of the stack

    public StackX(int s) {// constructor

            maxSize = s;      // set array size
            stackArray = new double[maxSize];
            top = -1;         // no items
    }
    public void push(double j) {

            // check whether stack is full
            if (top == maxSize – 1)
                System.out.println("Stack is full");
            else
                stackArray[++top] = j;
    }
```

```
public double pop() {
        // check whether stack is empty
        // if not
        // access item and decrement top
    }

public double peek() {

        // check whether stack is empty
        // if not
        // access item
    }
```

Implementation – pop/peek

```
class StackX {
    private int maxSize;    // size of stack array
    private double[] stackArray;
    private int top;        //top of the stack

    public StackX(int s) { // constructor

        maxSize = s;      // set array size
        stackArray = new double[maxSize];
        top = -1;         // no items
    }
    public void push(double j) {

        // check whether stack is full
        if (top == maxSize – 1)
            System.out.println("Stack is full");
        else
            stackArray[++top] = j;
    }
```

```
public double pop() {
        if (top == -1)
            return -99;
        else
            return stackArray[top--];
    }

public double peek() {
        if (top == -1)
            return -99;
        else
            return stackArray[top];
    }
```

Question

isEmpty() method returns true if the stack is empty and isFull() method return true if the Stack is full.
Implement isEmpty() and isFull() methods of the stack class.

Creating a stack

Question

Using the implemented StackX class, Write a program to create a stack with maximum size 10 and insert the following items to the stack.

30 80 100 25

Delete all the items from the stack and display the deleted items.

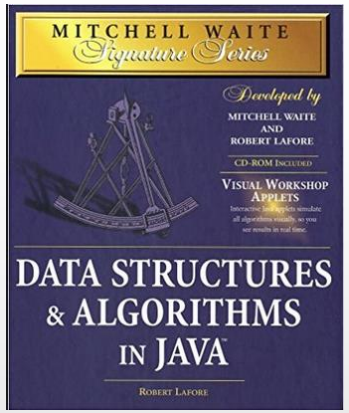
Creating a stack

```
class StackApp {
        public static void main(String[] args) {
                StackX  theStack = new StackX(10);  // create a stack with max size 10

                theStack.push(30);  // insert given items
                theStack.push(80);
                theStack.push(100);
                theStack.push(25);

                while( !theStack.isEmpty() ) {    // until it is empty, delete item from stack

                        double val = theStack.pop();
                        System.out.print(val);
                        System.out.print(" ");
                }
        }
} // end of class
```

References

1. Mitchell Waite,Robert Lafore, Data Structures and Algorithms in Java,2nd Edition, Waite Group Press,1998.